Lecture 8. Classification I

Supervised learning, naive Bayes, LDA and QDA, logistic regression and cross-entropy loss

Contents

8.1 Supervised learning **2**

8.2 Naive Bayes **3**

8.3 Discriminant analysis **3**

8.4 Logistic regression for classification **5**

8.4.1 Why not squared loss 5

8.4.2 Cross-entropy loss 6

8.1 Supervised learning

Motivation. Many business and scientific decisions are classifications: is this e-mail phishing, will this customer leave, will this user click? Classification is prediction with a categorical outcome, and the same logic of training on labelled data and checking on new data applies.

Data scientists are often asked to automate decisions: Is an e-mail a phishing attempt? Is a customer likely to churn? Will a user click on an ad? These are **classification** problems, a form of **supervised learning**: train a model on data where the outcome is known, then apply it where it is not. Machine learning algorithms predict, classify or cluster; prediction (numeric response) and classification (categorical response) are supervised, clustering (no response) is **unsupervised**.

Prediction. In `RailTrail`, a LOESS smoother of `volume` on `avgtemp` is a prediction model: the response "supervises" the fit, and the fitted values are predictions.

```
RailTrail %>%
  mutate(loess.fit = loess(volume ~ avgtemp, data = .)$fitted) %>%
  select(volume, loess.fit, avgtemp) %>% head(4)
```

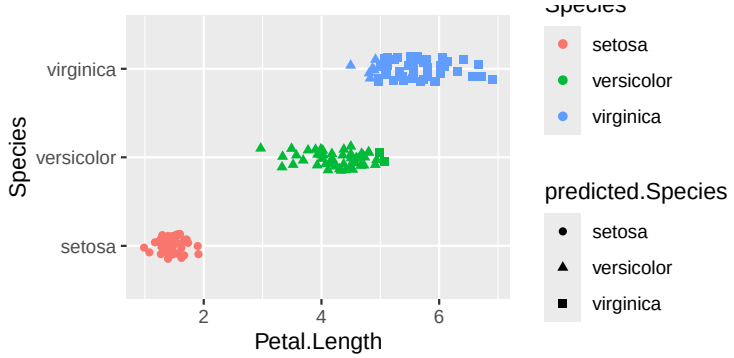
```
##   volume loess.fit avgtemp
## 1    501  469.4541    66.5
## 2    419  443.2781    61.0
## 3    397  462.1633    63.0
## 4    385  400.6197    78.0
```

Classification. In `iris`, petal length alone almost separates the three species. A crude rule read off a plot, setosa below 2, versicolor between 2 and 5, virginica above 5, already classifies well:

```
iris.crude <- iris %>%
  mutate(predicted.Species = ifelse(Petal.Length < 2, "setosa",
                                    ifelse(Petal.Length < 5, "versicolor", "virginica")))
iris.crude %>% count(Species, predicted.Species)
```

```
##      Species predicted.Species  n
## 1     setosa            setosa 50
## 2 versicolor        versicolor 48
## 3 versicolor         virginica  2
## 4  virginica        versicolor  6
## 5  virginica         virginica 44
```

```
ggplot(iris.crude, aes(Petal.Length, Species, colour = Species, shape = predicted.
    Species)) +
  geom_jitter(height = 0.15)
```



Classification is a special case of prediction: `predicted.Species` predicts `Species` and takes the same set of values. With two variables (petal length and width) we can still look; to use all four, or many more, we need model- or algorithm-based methods. Visualisation remains useful for exploration and for communicating results.

8.2 Naive Bayes

Motivation. Bayes' theorem turns probabilities of the predictors given the class, which are easy to estimate, into the probability of the class given the predictors, which is what we want. With many predictors one strong simplification makes this practical.

Example. The slides use the 1994 US Census (32 561 adults; 24% earn more than \$50 000). With $y = 1$ for high earners and $x = 1$ for male, $P(y = 1) = 0.241$, $P(\text{male}) = 0.670$ and $P(\text{male} \mid y = 1) = 0.845$, so by Bayes' theorem

$$P(y = 1 \mid \text{male}) = \frac{P(\text{male} \mid y = 1)\, P(y = 1)}{P(\text{male})} = \frac{0.845 \times 0.241}{0.670} \approx 0.30.$$

Marginal probabilities are usually easy to compute, so knowing $P(y \mid x)$ is equivalent to knowing $P(x \mid y)$. A classifier assigns x^* to $y = 1$ when $P(y = 1 \mid x^*) > P(y = 0 \mid x^*)$.

With several predictors, regression models $P(y \mid x_1, \ldots, x_p)$ directly; naive Bayes instead models $P(x_1, \ldots, x_p \mid y)$ under the (overly) simple assumption that the predictors are **conditionally independent** given the class:

$$P(x_1, \ldots, x_p \mid y) = \prod_{j=1}^{p} P(x_j \mid y),$$

each factor being easy to estimate since y takes few values. By Bayes' theorem, for classes $k = 1, \ldots, K$,

$$P(y = i \mid x_1, \ldots, x_p) = \frac{\prod_j P(x_j \mid y = i)\; P(y = i)}{\sum_{k=1}^{K} \prod_j P(x_j \mid y = k)\; P(y = k)}.$$

The labels $1, \ldots, K$ carry no order; numeric predictors need a model for $P(x_j \mid y)$, usually a normal density. There is no regression, only the conditional independence assumption, yet naive Bayes often works well.

The census data must be downloaded from the UCI repository, so here we use `ISLR2::Default` (10 000 credit-card customers; did they default?) for all classifiers in this lecture. Split 80/20 into training and test sets:

```
set.seed(364)
idx <- sample(nrow(Default), 0.8 * nrow(Default))
train <- Default[idx, ]; test <- Default[-idx, ]
mean(train$default == "Yes")
```

```
## [1] 0.03125
```

```
accuracy <- function(pred, truth) mean(pred == truth)
mod_nb <- naiveBayes(default ~ student + balance + income, data = train)
accuracy(predict(mod_nb, test), test$default)
```

```
## [1] 0.9625
```

Only about 3% default, so always predicting "No" would already be 97% accurate: accuracy alone is a poor measure with unbalanced classes (Lecture 9 introduces better ones) [Supplement].

8.3 Discriminant analysis

Motivation. If each class's predictors are roughly normal, Bayes' theorem gives the optimal rule in closed form. Linear discriminant analysis is that rule with a common covariance, and its decision boundaries are straight lines.

Discriminant analysis, introduced by R. A. Fisher in 1936, is the oldest statistical classifier. **Linear discriminant analysis** (LDA) is less common now that tree methods exist, but still appears in applications.

One predictor. Assume $p(x \mid y = k) = f_k(x) = \phi(x; \mu_k, \sigma^2)$, normal with class-specific means and a common variance, with prior $\pi_k = P(y = k)$. Plugging in the estimates $\hat\pi_k = n_k/n$, $\hat\mu_k$, $\hat\sigma^2$, the Bayes rule assigns x to the class with the largest **discriminant function**

$$\delta_k(x) = x\,\frac{\hat\mu_k}{\hat\sigma^2} - \frac{\hat\mu_k^2}{2\hat\sigma^2} + \log\hat\pi_k.$$

Proposition 8.1 (LDA discriminant)

Under the model above, $P(y = k \mid x)$ is largest for the k maximising $\delta_k(x)$. With two classes and $\pi_0 = \pi_1$, the boundary is $x = (\mu_0 + \mu_1)/2$.

Proof [Supplement]. $P(y = k \mid x) \propto \pi_k f_k(x)$, with the same denominator for all k. Taking logs, $\log\pi_k - \frac{(x-\mu_k)^2}{2\sigma^2} + \text{const} = \log\pi_k + \frac{x\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} - \frac{x^2}{2\sigma^2} + \text{const}$; the last two terms are common to all classes. With equal priors, $\delta_1(x) > \delta_0(x)$ iff $x(\mu_1 - \mu_0)/\sigma^2 > (\mu_1^2 - \mu_0^2)/(2\sigma^2)$, i.e. (for $\mu_1 > \mu_0$) iff $x > (\mu_0 + \mu_1)/2$. ■

Several predictors. Each class is multivariate normal $N(\mu_k, \Sigma)$ with a common covariance matrix and density

$$f(x) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\Big\{-\frac{1}{2}(x-\mu_k)^{\mathsf T}\Sigma^{-1}(x-\mu_k)\Big\}.$$

(Contours are circles when the variables are uncorrelated with equal variances, tilted ellipses when correlated.) Taking logs and dropping terms common to all classes gives

$$\delta_k(x) = x^{\mathsf T}\hat\Sigma^{-1}\hat\mu_k - \frac{1}{2}\hat\mu_k^{\mathsf T}\hat\Sigma^{-1}\hat\mu_k + \log\hat\pi_k,$$

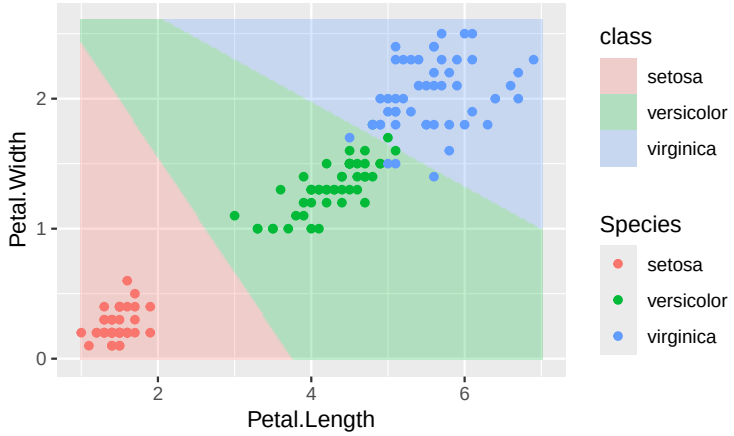
linear in x; LDA assigns x to the class with the largest $\delta_k(x)$, for any number of classes.

QDA. Allowing a different covariance Σ_k for each class gives **quadratic discriminant analysis**, with quadratic boundaries: more flexible, with more parameters. LDA often performs well in practice and similarly to logistic regression.

```
mod_lda <- lda(default ~ student + balance + income, data = train)
mod_qda <- qda(default ~ student + balance + income, data = train)
c(lda = accuracy(predict(mod_lda, test)$class, test$default),
  qda = accuracy(predict(mod_qda, test)$class, test$default))
```

```
##    lda    qda
## 0.9655 0.9680
```

```
lda2 <- lda(Species ~ Petal.Length + Petal.Width, data = iris)
grid <- expand_grid(Petal.Length = seq(1, 7, length = 200), Petal.Width = seq(0, 2.6,
    length = 200))
grid$class <- predict(lda2, grid)$class
ggplot(iris, aes(Petal.Length, Petal.Width)) +
  geom_raster(data = grid, aes(fill = class), alpha = 0.25) +
  geom_point(aes(colour = Species))
```



LDA on two iris measurements: the decision boundaries between the three classes are straight lines.

8.4 Logistic regression for classification

Motivation. Logistic regression (Lecture 6) gives probabilities; a threshold turns them into classes. The more interesting question is how its coefficients are fitted: least squares fails here, and the right loss, cross-entropy, turns out to be maximum likelihood.

The model $\operatorname{logit}(q) = \log \frac{q}{1-q} = x^\mathsf{T}\beta$ gives $q = P(y = 1 \mid x) = \operatorname{expit}(x^\mathsf{T}\beta) = 1/(1 + e^{-x^\mathsf{T}\beta})$. Classifying $y = 1$ when $q > 0.5$ means $x^\mathsf{T}\beta > 0$: the **decision boundary** is the hyperplane $x^\mathsf{T}\beta = 0$.

High earners. With capital gains (taxes paid on gains from assets, which indicate wealth and are part of income) as the only predictor, the slides obtain $\hat\beta_0 = -1.37$ and $\hat\beta_1 = 0.000335$; the boundary $-\hat\beta_0/\hat\beta_1 \approx \$4\,102$ means: classify a taxpayer as high-income if their capital gains exceed $4 102. The same on `Default` with balance:

```
mod_log_1 <- glm(default ~ balance, family = binomial, data = train)
tidy(mod_log_1)
```

```
## # A tibble: 2 x 5
##   term         estimate std.error statistic   p.value
##   <chr>           <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept) -10.8      0.421        -25.8 1.56e-146
## 2 balance       0.00559  0.000257      21.8 3.12e-105
```

```
-coef(mod_log_1)[1] / coef(mod_log_1)[2]
```

```
## (Intercept)
##     1939.69
```

A customer is classified as a defaulter when the balance exceeds about $1 940.

8.4.1 Why not squared loss

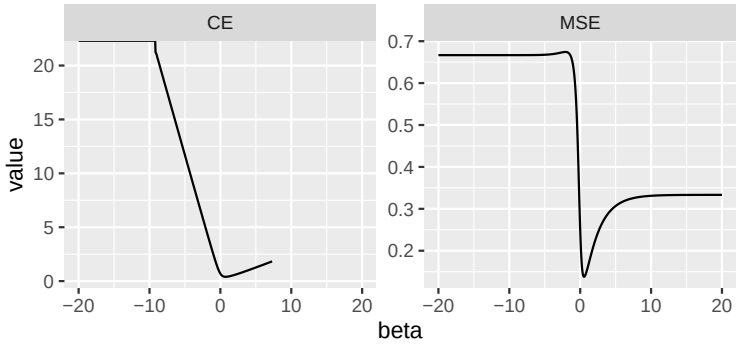
In linear regression the coefficients minimise the mean squared error. Applied to logistic regression,

$$\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \operatorname{expit}(x_i^\mathsf{T}\beta)\right)^2$$

compares a binary y with a probability, and is not convex in β. For a toy data set with a single coefficient ($\operatorname{logit} q = \beta x$):

```
x <- c(-4, -2, -0.5, 1, 3, 5); y <- c(0, 0, 1, 0, 1, 1)
expit <- function(z) 1 / (1 + exp(-z))
tibble(beta = seq(-20, 20, by = 0.05)) %>%
  mutate(MSE = map_dbl(beta, ~ mean((y - expit(.x * x))^2)),
         CE = map_dbl(beta, ~ -mean(y * log(expit(.x * x)) + (1 - y) * log(1 - expit(.x
    * x))))) %>%
  pivot_longer(-beta) %>%
  ggplot(aes(beta, value)) + geom_line() + facet_wrap(~ name, scales = "free_y")
```

```
## Warning: Removed 254 rows containing missing values or values outside the scale range
## (`geom_line()`).
```




```
mse <- function(b) mean((y - expit(b * x))^2)
sapply(c(-20, -10.34, -2, 0.54, 5), mse)
```

```
## [1] 0.6666667 0.6666720 0.6741982 0.1378328 0.3067829
```

```
optimize(mse, c(-5, 5))$minimum
```

```
## [1] 0.5446891
```

The MSE curve (right) is not convex: it has a hump near $\beta = -2$ and for negative β becomes an almost flat plateau at $2/3$. A gradient method started on the left keeps moving away and stops where the slope is numerically zero (the slides report such a point, $\beta = -10.34$), far from the global minimum $\beta \approx 0.54$. Adding variables does not fix this, and the squared error of a probability is bounded by 1, which is conceptually questionable.

8.4.2 Cross-entropy loss

For binary y and predicted probability q, the **cross-entropy** (log) loss is

$$-\{y \log q + (1 - y) \log(1 - q)\}.$$

If $y = 1$ the loss is $-\log q$, zero when $q \to 1$ and infinite when $q \to 0$; if $y = 0$ it is $-\log(1 - q)$. It strongly penalises confident wrong predictions and, averaged over the data,

$$-\frac{1}{n} \sum_{i=1}^{n} \left\{ y_i \log \operatorname{expit}(x_i^\mathsf{T} \beta) + (1 - y_i) \log \left(1 - \operatorname{expit}(x_i^\mathsf{T} \beta)\right) \right\},$$

is convex in β (left panel above), so it has a single minimum.

Convexity of cross-entropy [Supplement]

With $q_i = \operatorname{expit}(x_i^\mathsf{T} \beta)$, the gradient of the average loss is $\frac{1}{n} \sum_i (q_i - y_i) x_i$ and the Hessian is $\frac{1}{n} \sum_i q_i (1 - q_i) x_i x_i^\mathsf{T}$, a positive semidefinite matrix. Hence the loss is convex, and New-

ton's method (iteratively reweighted least squares, what `glm()` does) converges to the global minimum when it exists.

Connection to maximum likelihood. Given x_i, $y_i \sim \text{Bernoulli}(q_i)$ with likelihood $q_i^{y_i}(1-q_i)^{1-y_i}$, so the log-likelihood is

$$\sum_{i=1}^{n} \{y_i \log q_i + (1 - y_i) \log(1 - q_i)\},$$

exactly $-n$ times the average cross-entropy. Minimising cross-entropy *is* maximum likelihood estimation. Many "model + loss" combinations can be motivated this way: least squares (normal errors), and ridge and lasso (adding a prior on β).

More than two classes. Logistic regression extends to $K > 2$ classes, e.g. a chain of binary models ("1" vs "2 or 3", then "2" vs "3"), or the multinomial model (`nnet::multinom()`), but discriminant analysis is more often used for multiple classes.

Summary. Naive Bayes assumes conditional independence of the predictors; discriminant analysis (LDA, QDA) assumes multivariate normal predictors within classes; logistic regression assumes a linear model for logit $P(y = 1 \mid x)$, fitted by cross-entropy, i.e. maximum likelihood.